MCMP117 · Week 10 · Course Notes

Fundamentals of Operating Systems

File & Directory Operations

Create, copy, move, delete, view and link files — and know what the kernel does each time.

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain, in terms of directory entries, inodes and data blocks (Week 7), what `touch`, `cp`, `mv`, `ln`, `ln -s` and `rm` change.
2. Create directories and files with `mkdir` (`-p`) and `touch`, and read the three timestamps atime, mtime and ctime.
3. Copy, move, rename and delete files and directory trees with `cp`, `mv`, `rm`, `rmdir` — safely.
4. Use the wildcards `*`, `?`, `[ ]` and brace expansion `{a,b}`, `{1..5}`, and explain that the *shell* expands them.
5. Work with file names that contain spaces.
6. Inspect files with `cat`, `less`, `head`, `tail`, `wc`, `file`, `stat`, `du` and `df`.
7. Create hard and symbolic links and predict what happens when the original is deleted.

Contents

1 From Week 9 to Week 10 1

2 Names, Inodes and System Calls 2

3 Creating: `mkdir` and `touch` 3

4 Copying, Moving and Deleting 5

5 Wildcards and Brace Expansion 8

6 Names with Spaces: Quoting 10

7 Looking Inside Files 10

8 Links Revisited 13

9 Command Reference 15

10 Common Errors and What They Mean 16

11 Summary 17

12 Try It Yourself 17

13 Review Questions 17

14 Lab Sheet (graded, 100 points) 18

15 Lab Sheet Solutions 19

16 Looking Ahead: Week 11 22

1 From Week 9 to Week 10

In Week 9 we met the terminal and the shell, learned the anatomy of a command (`command -options arguments`), and moved around the Linux directory tree with `pwd`, `ls` and `cd`, using absolute paths (`/home/student/lab`) and relative paths (`docs`, `../data`). We only *looked*. This week we *change* the file system: we create, copy, move, rename, delete and link files and directories, and we look inside them.

Everything here rests on the file-system theory of Week 7, so we start with a short recap. Then come the commands, each shown with real output.

Setting up

All examples run in the practice folder created by the lab script (it is in the `code/` folder of every Linux week). Run it before each session; it deletes and re-creates *only* `~/lab`:

```
$ bash ~/mcmp117_lab_setup.sh
$ cd ~/lab
```

Every demo of the slides is a script in `code/` (for example `demo_cp_mv.sh`) and its captured output is the matching `out_*.txt` file. The prompt `student@okan-lab:~/lab$` shows the user, the machine and the current directory; you type only what follows the `$`.

> **Your output will differ slightly**
>
> The outputs were captured as the ordinary user `student` with `LC_ALL=C.UTF-8`. In other locales `ls` may *sort* differently (in the C locale capitals come first, so `IMG_014.JPG` is listed before `holiday.jpeg`). Dates, times (ours are in UTC, `+0000`; in Türkiye you will see `+0300`), inode numbers, disk sizes and user IDs will be different on your machine. Everything else should match.

2 Names, Inodes and System Calls

2.1 Recap: what is stored where

Remember from Week 7: a **directory** is a table of (name, inode number) pairs. The **inode** holds everything else about a file — type, size, owner, permissions, timestamps, **link count** and the pointers to the **data blocks** — but *not* its name. That is why one inode can have several names, and why some operations are instant while others copy data.

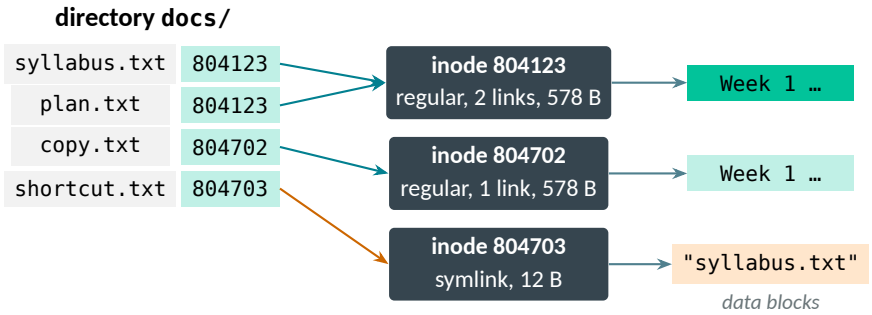


Figure 1: Real situation from the links demo (Section 8): `plan.txt` is a hard link, `copy.txt` a copy, `shortcut.txt` a symbolic link.

With this picture every command of the week can be described by *what it changes*:

Command	What changes	System call
`touch new`	a new name and a new (empty) inode	`openat(O_CREAT)`
`mkdir d`	a new name and a new directory inode	`mkdir()`
`cp a b`	a new name, a new inode *and* a copy of all data	`open/read/write`
`mv a b`	only directory entries; same inode, data untouched	`renameat2()`
`ln a b`	a new name pointing to the *same* inode; link count $+1$	`linkat()`
`ln -s a b`	a new name and a small inode whose data is the path a	`symlinkat()`
`rm a`	a name is removed; link count -1; data freed at 0	`unlinkat()`
`rmdir d`	an empty directory is removed	`rmdir()`

2.2 Commands are programs; their work is done by system calls

Each command is a small program in `/usr/bin`. The shell starts it with `fork()` and `exec()` (Week 3), and the program asks the kernel to do the real work through a **system call** (Week 2). With `strace` we can watch this; `-e trace=…` shows only the calls we name (script `demo_strace.sh`; the lines `+++ exited with 0 +++` after each command are omitted):

```
student@okan-lab:~/lab$ strace -e trace=mkdir,mkdirat mkdir newdir
mkdir("newdir", 0777)                   = 0
student@okan-lab:~/lab$ strace -e trace=rename,renameat,renameat2 mv newdir olddir
renameat2(AT_FDCWD, "newdir", AT_FDCWD, "olddir", RENAME_NOREPLACE) = 0
student@okan-lab:~/lab$ touch file1
student@okan-lab:~/lab$ strace -e trace=link,linkat ln file1 file2
linkat(AT_FDCWD, "file1", AT_FDCWD, "file2", 0) = 0
```

```
student@okan-lab:~/lab$ strace -e trace=symlink,symlinkat ln -s olddir sym
symlinkat("olddir", AT_FDCWD, "sym")    = 0
student@okan-lab:~/lab$ strace -e trace=unlink,unlinkat rm file1 file2 sym
unlinkat(AT_FDCWD, "file1", 0)          = 0
unlinkat(AT_FDCWD, "file2", 0)          = 0
unlinkat(AT_FDCWD, "sym", 0)            = 0
student@okan-lab:~/lab$ strace -e trace=rmdir,unlinkat rmdir olddir
rmdir("olddir")                         = 0
```

Observe:

- `mv` did not copy a single byte: it made one `renameat2()` call. The flag `RENAME_NOREPLACE` means "fail if the target exists"; `mv` uses it first and only replaces a target deliberately.
- `rm` with three names makes three `unlinkat()` calls. The kernel's name for deleting is *unlink*: it removes one *link* (name) of an inode.
- The `…at` versions take a directory as their first argument; `AT_FDCWD` means "relative to the current working directory" — exactly the relative paths of Week 9.

3 Creating: `mkdir` and `touch`

3.1 `mkdir`: make directories

`mkdir name` creates one empty directory. Two errors are common: the name is already taken, or a *parent* directory on the path does not exist. The option `-p` (*parents*) creates every missing directory on the path and is silent if the directory already exists (`demo_mkdir_touch.sh`):

```
student@okan-lab:~/lab$ mkdir test
student@okan-lab:~/lab$ ls -ld test
drwxrwxr-x 2 student student 4096 Oct 12 08:47 test
student@okan-lab:~/lab$ mkdir test
mkdir: cannot create directory ‘test’: File exists
student@okan-lab:~/lab$ mkdir trip/day1
mkdir: cannot create directory ‘trip/day1’: No such file or directory
student@okan-lab:~/lab$ mkdir -p trip/day1
student@okan-lab:~/lab$ mkdir -p work/2026/october
student@okan-lab:~/lab$ tree work trip
work
└── 2026
    └── october
trip
└── day1

5 directories, 0 files
```

`ls -ld test` lists the directory *itself* (`-d`), not its contents. Its link count is 2: the name `test` in `~/lab` and the entry `.` inside it. `tree` draws a directory tree; its final count includes the starting directories (5: `work`, `2026`, `october`, `trip`, `day1`).

3.2 `touch`: empty files and timestamps

`touch file` has two effects. If the file does not exist, it is created with size 0. If it exists, its content is left alone and only its timestamps are set to "now". With `-d` you can set any time:

```
student@okan-lab:~/lab$ touch todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Oct 12 08:47 todo.txt
student@okan-lab:~/lab$ touch -d "2026-01-15 09:30" todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Jan 15  2026 todo.txt
student@okan-lab:~/lab$ stat todo.txt
  File: todo.txt
  Size: 0               Blocks: 0          IO Block: 4096   regular empty file
Device: 254,0   Inode: 804708      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1010/ student)   Gid: ( 1012/ student)
Access: 2026-01-15 09:30:00.000000000 +0000
Modify: 2026-01-15 09:30:00.000000000 +0000
Change: 2026-10-12 08:47:56.442757013 +0000
 Birth: 2026-10-12 08:47:56.430757013 +0000
student@okan-lab:~/lab$ touch todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Oct 12 08:47 todo.txt
```

Note that `ls -l` shows `Jan 15 2026` — a year instead of a time of day — because the date is more than six months away from today. `stat` (Section 7.4) shows all four times kept in the inode.

3.3 Three (or four) timestamps

An inode keeps three times: **atime** (Access, last read of the data), **mtime** (Modify, last change of the data — the time `ls -l` shows) and **ctime** (Change, last change of the *inode*: data, link count, permissions, owner, times). **ctime is not the creation time**; newer file systems such as ext4 also record a creation time, shown by `stat` as *Birth*.

We verified on our test machine (ext4) which action changes which time:

Action	atime	mtime	ctime
read: `cat f`	yes (lazily)	—	—
write: `echo x >> f`	—	yes	yes
`touch f`	yes	yes	yes
rename: `mv f g`	—	—	yes
new hard link: `ln f h`	—	—	yes
permissions: `chmod` (Week 11)	—	—	yes

"Lazily": to avoid a disk write for every read, Linux mounts file systems with `relatime` and updates atime only when it is older than mtime or more than 24 hours old. Backup programs and `make` rely on mtime; `cp -p` keeps the original mtime of a file instead of giving the copy the current time.

4 Copying, Moving and Deleting

4.1 cp: copy

`cp SOURCE TARGET` copies a file. If `TARGET` is an existing directory, the copy keeps its name and goes inside it; otherwise `TARGET` is the new name. With several sources, the last argument must be a directory.

Options: `-v` (verbose) reports each copy, `-i` (interactive) asks before overwriting, `-r` (recursive) copies directories (`demo_cp_mv.sh`):

```
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/syllabus_copy.txt
student@okan-lab:~/lab$ ls backup
syllabus.txt  syllabus_copy.txt
student@okan-lab:~/lab$ cp -v docs/notes_01.txt docs/notes_02.txt backup/
'docs/notes_01.txt' -> 'backup/notes_01.txt'
'docs/notes_02.txt' -> 'backup/notes_02.txt'
student@okan-lab:~/lab$ cp -i docs/syllabus.txt backup/
cp: overwrite 'backup/syllabus.txt'? n
student@okan-lab:~/lab$ cp projects/alpha backup/
cp: -r not specified; omitting directory 'projects/alpha'
student@okan-lab:~/lab$ cp -r projects/alpha backup/
student@okan-lab:~/lab$ tree backup
backup
├── alpha
│   ├── README.md
│   ├── docs
│   │   └── design.txt
│   └── src
│       └── main.c
├── notes_01.txt
├── notes_02.txt
├── syllabus.txt
└── syllabus_copy.txt

4 directories, 7 files
student@okan-lab:~/lab$ ls -i docs/syllabus.txt backup/syllabus.txt
804330 backup/syllabus.txt  804123 docs/syllabus.txt
```

Observations:

- `cp -i` asked before overwriting; we answered `n`. (In the script the answer is supplied by `echo n`; in a terminal you type it.) **Without** `-i`, `cp` overwrites an existing file silently.
- A directory is not copied without `-r`. *Recursive* means: copy the directory, then do the same for every entry in it, and so on down the tree.
- `ls -i` shows *different* inode numbers (804330 and 804123): a copy is a new, independent file. Changing one does not change the other.

Useful extras: `cp -p` also keeps times and permissions; `cp -a` (archive) is `-r` plus everything that `-p` keeps, and is what you want for backups.

> **A silent trap**
>
> If `backup/` does *not* exist, `cp notes.txt backup` creates a *file* named `backup`. Writing `backup/` with a trailing slash makes `cp` fail with an error instead.

4.2 `mv`: rename and move

Linux has no separate rename command for single files: `mv` does both. Inside one directory `mv old new` renames; `mv file dir/` moves the name into another directory. Directories are moved the same way,

without `-r`:

```
student@okan-lab:~/lab$ mv backup/syllabus.txt backup/syllabus_old.txt
student@okan-lab:~/lab$ ls -i backup/syllabus_old.txt
804330 backup/syllabus_old.txt
student@okan-lab:~/lab$ mv -v docs/notes_03.txt backup/
renamed 'docs/notes_03.txt' -> 'backup/notes_03.txt'
student@okan-lab:~/lab$ mv backup/alpha backup/alpha_2026
student@okan-lab:~/lab$ ls docs backup
backup:
alpha_2026    notes_02.txt  syllabus_copy.txt
notes_01.txt  notes_03.txt  syllabus_old.txt

docs:
'final report.txt'   notes_01.txt   syllabus.txt
 linux_quotes.txt    notes_02.txt
```

The inode number of `backup/syllabus.txt` was 804330 before the rename and is still 804330 after it: only the directory entry changed (one `renameat2()` call, Section 2). Hence `mv` within one file system is instant whatever the file size. Between two file systems (for example from the disk to a USB stick) the inode cannot move, so `mv` copies the data and then deletes the original. Like `cp`, `mv` overwrites an existing target silently; use `mv -i`.

4.3 `rm` and `rmdir`: delete

`rm` removes names; `rmdir` removes *empty* directories (`demo_rm.sh`):

```
student@okan-lab:~/lab$ touch junk1.txt junk2.txt junk3.txt
student@okan-lab:~/lab$ rm junk1.txt
student@okan-lab:~/lab$ rm -i junk2.txt
rm: remove regular empty file 'junk2.txt'? y
student@okan-lab:~/lab$ rm -v junk3.txt
removed 'junk3.txt'
student@okan-lab:~/lab$ ls junk*
ls: cannot access 'junk*': No such file or directory
student@okan-lab:~/lab$ mkdir empty_dir
student@okan-lab:~/lab$ rmdir empty_dir
student@okan-lab:~/lab$ rmdir projects/beta
rmdir: failed to remove 'projects/beta': Directory not empty
student@okan-lab:~/lab$ rm projects/beta
rm: cannot remove 'projects/beta': Is a directory
student@okan-lab:~/lab$ cp -r projects/beta beta_copy
student@okan-lab:~/lab$ tree beta_copy
...
3 directories, 3 files
student@okan-lab:~/lab$ rm -rv beta_copy
removed 'beta_copy/src/main.c'
removed directory 'beta_copy/src'
removed 'beta_copy/README.md'
removed 'beta_copy/docs/design.txt'
removed directory 'beta_copy/docs'
removed directory 'beta_copy'
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
```

- `rm junk1.txt` prints nothing and asks nothing.

- `rm -i` asks ("remove regular empty file?"); `rm -v` reports what it removed.
- `ls junk*` fails because no name matches the pattern any more; the shell then passes the pattern unchanged (Section 5).
- `rmdir` refuses a non-empty directory; `rm` without `-r` refuses any directory.
- `rm -rv` shows the order of work: every directory is emptied first, because the kernel's `rmdir()` only removes empty directories.

Warning: rm is forever

There is **no recycle bin** on the command line. `rm` calls `unlinkat()`; when the link count reaches 0 (and no program has the file open) the blocks are marked free and soon reused. The "Trash" of a desktop is a feature of the file manager, not of the kernel.

- Before `rm` with a pattern, run `ls` with the *same* pattern; check where you are with `pwd`.
- `rm -r` deletes a whole tree. A stray space is enough for disaster: `rm -r ~/lab /old` deletes `~/lab` *and* tries `/old`.
- `rm -f` (force) never asks and hides errors. Never run `rm -rf` with `sudo` (Week 11) unless you understand every character.

If you damage `~/lab`, just run the setup script again. On a real system, backups are the only undo.

5 Wildcards and Brace Expansion

5.1 Who expands *? The shell

When you type `ls *.jpg`, the program `ls` never sees the star. *Before* starting the command, bash reads the current directory, finds all names that match the pattern, sorts them, and puts them on the command line as separate arguments. This is called **globbing** (pathname expansion). Then `fork()` and `exec()` start `ls` with the finished list (Week 3).

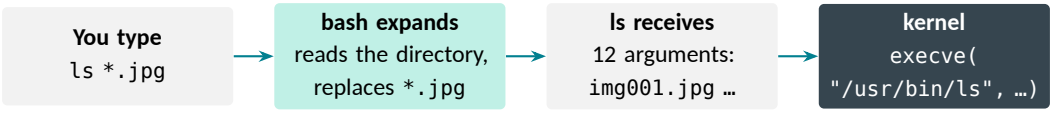


Figure 2: Wildcards are expanded by the shell, not by the command.

Three consequences:

1. `echo` shows exactly what a command will receive, so it is the perfect tool to test a pattern.
2. Every command "supports" wildcards, because no command ever sees them.
3. If nothing matches, bash leaves the pattern unchanged; the command then looks for a file literally named `junk*` and reports that it does not exist.

5.2 The patterns

Pattern	Matches	Example	Result in `photos/`
`*`	any string, also the empty one	`*.jpg`	12 names, img001–img012
`?`	exactly one character	`img00?.jpg`	9 names, img001–img009
`[0-2]`	one character from a range	`img01[0-2].jpg`	img010, img011, img012
`[135]`	one of the listed characters	`img00[135].jpg`	img001, img003, img005
`[!0]`	one character *not* in the set	`img0[!0]*`	img010 … img013

Two rules: Linux names are *case-sensitive* (`*.jpg` does not match `IMG_014.JPG`), and `*` does not match a leading dot, so hidden files are excluded unless you write the dot yourself. The script `demo_wildcards.sh` works in `~/lab/photos`, which contains 15 empty “photos”:

```
student@okan-lab:~/lab$ cd photos
student@okan-lab:~/lab/photos$ ls
IMG_014.JPG   img002.jpg  img005.jpg  img008.jpg  img011.jpg
holiday.jpeg  img003.jpg  img006.jpg  img009.jpg  img012.jpg
img001.jpg    img004.jpg  img007.jpg  img010.jpg  img013.png
student@okan-lab:~/lab/photos$ echo *.png
img013.png
student@okan-lab:~/lab/photos$ ls img00?.jpg
img001.jpg  img003.jpg  img005.jpg  img007.jpg  img009.jpg
img002.jpg  img004.jpg  img006.jpg  img008.jpg
student@okan-lab:~/lab/photos$ ls img01[0-2].jpg
img010.jpg  img011.jpg  img012.jpg
student@okan-lab:~/lab/photos$ ls *.[jJ][pP]*
IMG_014.JPG   img002.jpg  img005.jpg  img008.jpg  img011.jpg
holiday.jpeg  img003.jpg  img006.jpg  img009.jpg  img012.jpg
img001.jpg    img004.jpg  img007.jpg  img010.jpg
```

In a terminal, `ls` prints names in columns, sorted down each column. `*.[jJ][pP]*` matches every name whose extension starts with j/J followed by p/P: the twelve `.jpg`, `holiday.jpeg` and `IMG_014.JPG`.

5.3 No match, quoting and hidden files

```
student@okan-lab:~/lab/photos$ echo *.gif
*.gif
student@okan-lab:~/lab/photos$ echo "*.jpg"
*.jpg
```

```
student@okan-lab:~/lab/photos$ cd ../docs
student@okan-lab:~/lab/docs$ echo *
final report.txt linux_quotes.txt notes_01.txt notes_02.txt notes_03.txt syllabus.txt
student@okan-lab:~/lab/docs$ echo .*
.settings
```

There is no `.gif` file, so `*.gif` stays as it is. Quotes switch expansion off. In `docs/`, `*` gives all names except `.settings`; `.*` finds it. (Note that `final report.txt` is *one* argument containing a space; `echo` just prints the arguments separated by spaces.)

5.4 Brace expansion

Braces are a second, different expansion of bash. They do not look at the disk at all: they *generate* text. A list `{a,b,c}` produces each alternative; a range `{1..5}` produces a sequence; `{01..03}` keeps the leading zeros. Several braces in one word multiply:

```
student@okan-lab:~/lab/photos$ echo {1..5}
1 2 3 4 5
student@okan-lab:~/lab/photos$ echo img{01..03}.jpg
img01.jpg img02.jpg img03.jpg
student@okan-lab:~/lab/photos$ echo report_{draft,final}.txt
report_draft.txt report_final.txt
student@okan-lab:~/lab/photos$ mkdir -p ../trip/{day1,day2,day3}/{photos,notes}
student@okan-lab:~/lab/photos$ tree ../trip
../trip
├── day1
│   ├── notes
│   └── photos
├── day2
│   ├── notes
│   └── photos
└── day3
    ├── notes
    └── photos

10 directories, 0 files
```

Counting directories created by braces

`mkdir -p ../trip/{day1,day2,day3}/{photos,notes}` produces $3 \times 2 = 6$ paths. Together with `trip` and the 3 day directories created by `-p`, the tree contains

$$1 + 3 + 3 \times 2 = 10 \text{ directories},$$

exactly what `tree` reports. In general, braces with $n_1, n_2, \ldots$ alternatives produce $n_1 \cdot n_2 \cdots$ words.

In short: wildcards *find* existing names (sorted; unchanged if nothing matches), braces *make* new names (in the order written, whether or not files exist). Both are done by bash before the command starts, and both are switched off by quotes. A space after the comma (`{a, b}`) breaks brace expansion.

6 Names with Spaces: Quoting

The shell splits the command line at spaces *before* the command starts. A name with a space therefore becomes two arguments unless you protect it (`demo_quoting.sh`):

```
student@okan-lab:~/lab$ ls docs
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ cat docs/final report.txt
cat: docs/final: No such file or directory
cat: report.txt: No such file or directory
student@okan-lab:~/lab$ cat "docs/final report.txt"
Final report draft.
This file name contains a space.
```

```
student@okan-lab:~/lab$ cat 'docs/final report.txt'
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ cat docs/final\ report.txt
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ mkdir my photos
mkdir: cannot create directory ‘photos’: File exists
student@okan-lab:~/lab$ mkdir "my photos"
student@okan-lab:~/lab$ ls -d my*
 my  'my photos'
```

- `cat` received `docs/final` and `report.txt`: two errors.
- Double quotes `"…"`, single quotes `'…'` or a backslash before the space (`final\ report.txt`) all give one argument. The difference between `"` and `'` matters in Week 14 (variables are expanded inside double quotes only). Tab completion inserts the backslash for you.
- `mkdir my photos` created `my` and failed on `photos`, which already existed.
- `ls` prints names with spaces in quotes (`'my photos'`) so you can copy them back into a command.

Advice: in names you create yourself, use _ or - instead of spaces. (Clean-up: `rmdir my "my photos"`.)

7 Looking Inside Files

7.1 `cat`, `head`, `tail`

`cat` (*concatenate*) prints whole files; with several files it prints them one after another, and `-n` numbers the lines. `head -n N` and `tail -n N` print the first and last N lines (default 10) (`demo_view.sh`):

```
student@okan-lab:~/lab$ cat docs/linux_quotes.txt
Talk is cheap. Show me the code.
Given enough eyeballs, all bugs are shallow.
Most good programmers do programming not because they expect to get paid.
Theory and practice sometimes clash. And when that happens, theory loses.
Intelligence is the ability to avoid doing work, yet getting the work done.
student@okan-lab:~/lab$ cat -n docs/notes_01.txt docs/notes_02.txt
     1  Meeting notes 1
     2  Meeting notes 2
student@okan-lab:~/lab$ head -n 3 data/students.csv
id,name,department,midterm,final
2026001,Elif Yilmaz,ECT,96,99
2026002,Zeynep Dogan,ME,95,36
student@okan-lab:~/lab$ tail -n 3 logs/system.log
2026-10-12 14:27:00 INFO  user login
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
```

7.2 `tail -f` and `less`

`tail -f` (*follow*) keeps the file open and prints new lines as other programs append them — the standard way to watch a log file. Open two terminals: in the first run `tail -n 2 -f logs/system.log`, in the

second append a line:

```
$ echo "2026-10-12 15:00:00 WARN  disk almost full" >> logs/system.log
```

The new line appears at once in the first terminal; stop `tail` with Ctrl+C (shown as ^C). In our script a background job plays the role of the second terminal:

```
student@okan-lab:~/lab$ tail -n 2 -f logs/system.log
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
2026-10-12 15:00:00 WARN  disk almost full
^C
```

For long files, `cat` scrolls past too fast. `less file` shows one screen at a time and opens even huge files instantly, because it reads only what it displays. The `man` pages of Week 9 are shown with `less` too:

Key	Action	Key	Action
`Space / b`	next / previous page	`/word`	search forward
`↓ ↑`	one line	`n N`	next / previous match
`g G`	start / end of file	`q`	quit

7.3 `wc` and `file`

```
student@okan-lab:~/lab$ wc docs/linux_quotes.txt
  5  50 302 docs/linux_quotes.txt
student@okan-lab:~/lab$ wc -l data/*.csv data/*.txt
  41 data/students.csv
  30 data/cities.txt
  20 data/numbers.txt
  91 total
student@okan-lab:~/lab$ file README.txt src/hello.c data/students.csv photos/img001.jpg
    projects /usr/bin/ls
README.txt:        ASCII text
src/hello.c:       C source, ASCII text
data/students.csv: CSV ASCII text
photos/img001.jpg: empty
projects:          directory
/usr/bin/ls:       ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
     interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=
    daa5130f2a41f7fbc8662048f3294f3d439ca7ff, for GNU/Linux 3.2.0, stripped
```

wc (*word count*) prints lines, words and bytes: `linux_quotes.txt` has 5 lines, 50 words, 302 bytes. `-l`, `-w`, `-c` select one count. `students.csv` has 41 lines: 1 header line $+40$ students.

`file` ignores the name and examines the content, in particular the first bytes (the *magic number*, Week 7). Our “photos” are empty files with a `.jpg` name, so `file` says `empty`; a real JPEG starts with the bytes FF D8 FF. `/usr/bin/ls` is an ELF executable (the long line wraps).

7.4 `stat`: read the inode

`stat` prints the inode of Week 7 field by field (`demo_stat_du_df.sh`):

```
student@okan-lab:~/lab$ stat docs/syllabus.txt
```

```
  File: docs/syllabus.txt
  Size: 578             Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 804123      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1010/ student)   Gid: ( 1012/ student)
Access: 2026-10-12 08:47:57.251900492 +0000
Modify: 2026-10-12 08:47:57.254669759 +0000
Change: 2026-10-12 08:47:57.254669759 +0000
 Birth: 2026-10-12 08:47:57.251900492 +0000
student@okan-lab:~/lab$ stat -c "%n: %s bytes, inode %i, %h link(s)" docs/*.txt
docs/final report.txt: 53 bytes, inode 804125, 1 link(s)
docs/linux_quotes.txt: 302 bytes, inode 804124, 1 link(s)
docs/notes_01.txt: 16 bytes, inode 804127, 1 link(s)
docs/notes_02.txt: 16 bytes, inode 804128, 1 link(s)
docs/notes_03.txt: 16 bytes, inode 804129, 1 link(s)
docs/syllabus.txt: 578 bytes, inode 804123, 1 link(s)
```

- `Size:` 578 bytes of data; `Blocks:` 8 counts 512-byte units: $8 \times 512 = 4096$ B, i.e. one 4 KiB file-system block (`IO Block`). Even a 578-byte file occupies a whole block.
- `Inode` and `Links`: the inode number and the link count.
- `Access: (0664/-rw-rw-r--)`: the permissions in octal and symbolic form — next week's topic. `Uid`/`Gid`: owner and group.
- `stat -c` prints only chosen fields: `%n` name, `%s` size, `%i` inode, `%h` hard links, `%y` mtime.

7.5 `du` and `df`: how much space?

```
student@okan-lab:~/lab$ du -sh docs data logs photos
32K     docs
16K     data
36K     logs
4.0K    photos
student@okan-lab:~/lab$ du -h projects
8.0K    projects/beta/src
8.0K    projects/beta/docs
24K     projects/beta
8.0K    projects/alpha/src
8.0K    projects/alpha/docs
24K     projects/alpha
52K     projects
student@okan-lab:~/lab$ df -h ~
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda        252G   13G   30G  29% /
```

`du` (*disk usage*) adds up the blocks used by files and directories; `-s` prints one total per argument and `-h` uses K, M, G. `df` (*disk free*) reports per *file system* (per mounted disk), not per directory.

Checking du by hand

Each directory itself uses one 4 KiB block. `logs/` holds `access.log` ($23\,604$ B) and `system.log` ($5\,207$ B). Whole blocks are allocated:

$$\left\lceil \tfrac{23\,604}{4096} \right\rceil = 6 \text{ blocks} = 24 \text{ KiB}, \qquad \left\lceil \tfrac{5\,207}{4096} \right\rceil = 2 \text{ blocks} = 8 \text{ KiB},$$

so `logs` uses $4 + 24 + 8 = 36$ KiB, as du reports. `photos/` uses only 4.0 K: its 15 files are empty and need no data blocks at all. `projects/` is $4 + 2 \times 24 = 52$ K.

In `df`, `Use%` is Used$/($Used $+$ Avail$)$, rounded up: $12/(12+30) \approx 0.286$, shown as 29%. `Size` is larger than Used $+$ Avail because part of the disk is reserved and not available to ordinary users (on a normal ext4 disk 5% for root; on our virtual test machine much more).

8 Links Revisited

In Week 7 we created hard and symbolic links. Now we compare them directly with a copy, in `~/lab/docs` (`demo_links.sh`):

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ ln syllabus.txt plan.txt
student@okan-lab:~/lab/docs$ cp syllabus.txt copy.txt
student@okan-lab:~/lab/docs$ ln -s syllabus.txt shortcut.txt
student@okan-lab:~/lab/docs$ ls -li syllabus.txt plan.txt copy.txt shortcut.txt
804702 -rw-rw-r-- 1 student student 578 Oct 12 08:47 copy.txt
804123 -rw-rw-r-- 2 student student 578 Oct 12 08:47 plan.txt
804703 lrwxrwxrwx 1 student student  12 Oct 12 08:47 shortcut.txt -> syllabus.txt
804123 -rw-rw-r-- 2 student student 578 Oct 12 08:47 syllabus.txt
student@okan-lab:~/lab/docs$ echo "Week 16  Party" >> plan.txt
student@okan-lab:~/lab/docs$ tail -n 1 syllabus.txt copy.txt
==> syllabus.txt <==
Week 16  Party

==> copy.txt <==
Week 15 Final exam
student@okan-lab:~/lab/docs$ rm syllabus.txt
student@okan-lab:~/lab/docs$ ls -li plan.txt shortcut.txt
804123 -rw-rw-r-- 1 student student 593 Oct 12 08:47 plan.txt
804703 lrwxrwxrwx 1 student student  12 Oct 12 08:47 shortcut.txt -> syllabus.txt
student@okan-lab:~/lab/docs$ cat shortcut.txt
cat: shortcut.txt: No such file or directory
```

- `syllabus.txt` and `plan.txt`: the *same* inode (804123) with link count 2 — one file, two names (Figure 1).
- `copy.txt` has its own inode: a separate file with the same bytes.
- `shortcut.txt` has type `l`, its own inode, and size 12 = the length of the string `syllabus.txt` it stores.
- A line appended through `plan.txt` appears at the end of `syllabus.txt`, but not in `copy.txt`. The size grows from 578 to $593 = 578 + 15$ bytes (`Week 16  Party` is 14 characters plus a newline).
- After `rm syllabus.txt` the data is still reachable as `plan.txt` (link count back to 1), while the

symbolic link *dangles*: the path it stores no longer exists.

Symbolic links may point to directories; hard links to directories are forbidden, because they could create cycles in the directory graph (Week 7, general-graph directories):

```
student@okan-lab:~/lab/docs$ cd ..
student@okan-lab:~/lab$ ln -s projects/alpha/docs alpha_docs
student@okan-lab:~/lab$ ls -l alpha_docs
lrwxrwxrwx 1 student student 19 Oct 12 08:47 alpha_docs -> projects/alpha/docs
student@okan-lab:~/lab$ ls alpha_docs/
design.txt
student@okan-lab:~/lab$ ln projects alpha_hard
ln: projects: hard link not allowed for directory
student@okan-lab:~/lab$ ls -ld projects projects/alpha
drwxrwxr-x 4 student student 4096 Oct 12 08:47 projects
drwxrwxr-x 4 student student 4096 Oct 12 08:47 projects/alpha
```

The link count of a directory is $2 + (\text{number of subdirectories})$: its name in the parent, its own `.` entry, and the `..` entry of each child. `projects` contains `alpha` and `beta`, so $2 + 2 = 4$.

	Hard link `ln`	**Symbolic link `ln -s`**	**Copy `cp`**
Inode	same as the target	its own (stores a path)	its own
Data	shared	reached through the path	duplicated
Target deleted	data stays (count -1)	link dangles	copy unaffected
Directories	not allowed	allowed	with `-r`
Other file system	not possible	possible	possible

A relative symbolic link is interpreted relative to the directory that *contains the link*, not to your current directory. the command `ln -s syllabus.txt docs/shortcut.txt`, typed in `~/lab`, would therefore point to `docs/syllabus.txt` — a frequent source of confusion.

8.1 Hands-on: the same operations in C

The program `code/fileops.c` calls the system calls directly, one per command of this week. The macro CHECK prints the reason with `perror()` when a call fails, just like the commands print their error messages. Core of the program:

```c
CHECK(mkdir("box", 0775));                                  /* mkdir box         */
CHECK(fd = open("box/a.txt", O_WRONLY | O_CREAT, 0664)); /* touch box/a.txt   */
CHECK(write(fd, "hello\n", 6));  CHECK(close(fd));
CHECK(link("box/a.txt", "box/b.txt"));                      /* ln a.txt b.txt    */
CHECK(symlink("a.txt", "box/s.txt"));                       /* ln -s a.txt s.txt */
CHECK(rename("box/a.txt", "box/c.txt"));                    /* mv a.txt c.txt    */
CHECK(unlink("box/b.txt"));                                 /* rm b.txt          */
if (rmdir("box") == -1) perror("rmdir(\"box\")");           /* not empty: fails  */
CHECK(unlink("box/c.txt"));  CHECK(unlink("box/s.txt")); /* rm c.txt s.txt    */
CHECK(rmdir("box"));                                        /* rmdir box: now OK */
```

```
student@okan-lab:~/lab$ gcc -Wall -O2 fileops.c -o fileops
student@okan-lab:~/lab$ ./fileops
```

```
after open(O_CREAT):
  box/a.txt  inode 804677  links 1
after link() and symlink():
  box/a.txt  inode 804677  links 2
  box/b.txt  inode 804677  links 2
  box/s.txt  inode 804678  links 1
after rename() a.txt -> c.txt:
  box/c.txt  inode 804677  links 2
after unlink() b.txt:
  box/c.txt  inode 804677  links 1
rmdir("box"): Directory not empty
box removed
```

After `link()` both names show the same inode with 2 links; `rename()` keeps the inode; after `unlink()` the count is back to 1. `rmdir()` on the non-empty `box` fails with `Directory not empty` — the same text the `rmdir` command printed in the shell, because both come from the kernel's error code `ENOTEMPTY`. Compile it yourself with `gcc -Wall -O2 fileops.c -o fileops` (or `make`) and run it inside `~/lab` (`demo_fileops.sh`).

9 Command Reference

Command	What it does	Useful options
`mkdir dir`	create a directory	`-p` whole path, no error if it exists; `-v` verbose
`touch file`	create an empty file or update its times	`-d "2026-01-15 09:30"` set a time
`cp src dst`	copy files (new inode)	`-r` directories, `-i` ask, `-v` verbose, `-p` keep times, `-a` archive
`mv src dst`	rename or move (same inode)	`-i` ask before overwriting, `-v` verbose
`rm file`	remove names — no undo	`-i` ask, `-r` recursive, `-v` verbose, `-f` force (dangerous)
`rmdir dir`	remove an empty directory	`-p` also empty parents
`ln target name`	hard link	check with `ls -li`
`ln -s target name`	symbolic link	`ls -l` shows `name -> target`
`cat file…`	print / concatenate files	`-n` number lines
`less file`	page through a file	Space, b, /word, n, g, G, q
`head file`	first lines	`-n N`
`tail file`	last lines	`-n N`; `-f` follow
`wc file`	lines, words, bytes	`-l -w -c`
`file name`	identify content type	—
`stat file`	show the inode	`-c "%n %s %i %h %y"` chosen fields
`du dir`	disk space used	`-s` summary, `-h` human-readable
`df`	free space per file system	`-h`; `df -h ~` for your home
`tree dir`	draw a directory tree	`-d` directories only
`echo pattern`	show what the shell expands	—

10 Common Errors and What They Mean

All messages below are real outputs from this week's demos.

Message	Meaning and fix
`cp: -r not specified; omitting directory 'projects/alpha'`	You tried to copy a directory. Use `cp -r`.
`rm: cannot remove 'projects/beta': Is a directory`	`rm` without `-r` refuses directories. Use `rmdir` (if empty) or, carefully, `rm -r`.
`rmdir: failed to remove 'projects/beta': Directory not empty`	`rmdir` only removes empty directories. Empty it first, or use `rm -r` after checking with `ls`.
`mkdir: cannot create directory ‘trip/day1’: No such file or directory`	A parent directory (`trip`) is missing. Use `mkdir -p`.
`mkdir: cannot create directory ‘test’: File exists`	The name is taken (by a file or directory). `mkdir -p` is silent if it is a directory.
`cat: docs/final: No such file or directory` + `cat: report.txt: …`	Two errors for one file name: the name contains a space. Quote it.
`ls: cannot access 'junk*': No such file or directory`	The pattern matched nothing, so the shell passed it unchanged. Check the pattern with `echo`.
`ln: projects: hard link not allowed for directory`	Use a symbolic link: `ln -s`.
`cat: shortcut.txt: No such file or directory` (but `ls` shows it)	The symbolic link exists, its target does not: it dangles. Check with `ls -l`.
No message at all	The most dangerous case: `cp` and `mv` overwrite an existing target, and `rm` deletes, without a word. Use `-i` when unsure.

11 Summary

Week 10 in seven sentences

1. A directory maps names to inodes; `cp` creates a new inode with copied data, `mv` only changes names, `ln` adds a name, `ln -s` stores a path, `rm` removes a name and the data is freed when the link count reaches 0.
2. Each command is a program that performs one or a few system calls: `mkdir()`, `renameat2()`, `linkat()`, `symlinkat()`, `unlinkat()`, `rmdir()`.
3. `mkdir -p` creates whole paths; `touch` creates empty files and sets times; atime, mtime and ctime record access, data change and inode change.
4. `cp -r` copies trees; `cp` and `mv` overwrite silently unless `-i` is given; `rm` has no undo.
5. The shell expands wildcards (matching existing names) and braces (generating text) before the command runs; `echo` shows the result and quotes switch it off.
6. Names with spaces must be quoted or escaped.
7. `cat`, `less`, `head`, `tail -f`, `wc`, `file`, `stat`, `du` and `df` inspect files without changing them.

12 Try It Yourself

Short experiments

1. Run `stat` on a file, `cat` it, and run `stat` again. Did atime change? Why might it not change a second time?
2. Make a 1 MB file with `head -c 1000000 /dev/zero > big`, then compare `ls -l big` with `du -h big`.
3. Run `strace -e trace=openat,read,write cp docs/syllabus.txt /tmp/x.txt` and find the `read()` and `write()` calls that do the copying.

13 Review Questions

Conceptual

1. Why is `mv` of a 4 GB file instant within one disk but slow to a USB stick?
2. What is the difference between `rm` and `rmdir`? Why must `rm -r` empty a directory before removing it?
3. Who expands `*.txt`, the shell or the command? Give one experiment that proves it.
4. Explain the difference between wildcards and brace expansion with one example each.
5. What is ctime, and why is it *not* the creation time?
6. Compare a hard link, a symbolic link and a copy after the original name is deleted.

Numerical

7. How many directories does `mkdir -p site/{css,js,img}/{v1,v2}` create, including `site`?
8. A file has $10\,000$ bytes; the block size is 4096 B. How many blocks does it use, and what does `stat` show as `Blocks`?
9. A directory has 5 subdirectories and 20 files. What is its link count?
10. In `~/lab/photos` (fresh lab), how many names does `img0?[02468].*` match?
11. `df` shows Used $= 45$ G and Avail $= 55$ G. What is `Use%`?

Answers to the numerical questions

7. $1 + 3 + 3 \times 2 = 10$ directories.
8. $\lceil 10\,000/4096 \rceil = 3$ blocks of 4 KiB $= 12\,288$ B; `stat` counts 512-byte units: $3 \times 8 = 24$.
9. $2 + 5 = 7$; files do not add to a directory's link count.
10. The fourth character must be 0 or 1 and the fifth even: img002, 004, 006, 008 (fourth char 0) and img010, 012 (fourth char 1): 6 names.

11. $45/(45+55) = 0.45$, i.e. 45%.

14 Lab Sheet (graded, 100 points)

Instructions. Start from a fresh lab: `bash ~/mcmp117_lab_setup.sh && cd ~/lab`. Do the exercises *in order* (later ones use the results of earlier ones). For each exercise write the command(s) you used and the key lines of output; where a question asks "why", answer in one sentence. Before any `mv` or `rm` with a pattern, first run `ls` with the same pattern.

Part A --- Basic (33 points)

1. **(4)** With *one* command create the nested directories `practice/a/b/c`. Show the result with `tree`.
2. **(4)** With *one* command create three empty files `one.txt`, `two.txt`, `three.txt` inside `practice`. List `practice`.
3. **(5)** Copy `docs/syllabus.txt` into `backup/` under the name `syllabus_backup.txt`; verify with `ls -l backup`.
4. **(5)** Rename it to `syllabus_v1.txt`. Show with `ls -i` before and after that the inode number did not change, and explain why.
5. **(5)** Display the first 4 and the last 2 lines of `docs/syllabus.txt`.
6. **(5)** How many lines, words and bytes does `docs/linux_quotes.txt` have? How many *students* are in `data/students.csv`?
7. **(5)** Display `docs/final report.txt` using two different ways of quoting.

Part B --- Intermediate (39 points)

8. **(6)** Without changing directory, list the photos `img001.jpg`–`img009.jpg` using `?`, then `img010.jpg`–`img012.jpg` using `[ ]`, then show with `echo` which PNG files exist.
9. **(8)** Organise `photos/`: create the subdirectories `jpg`, `png` and `other`, move all `.jpg` files to `jpg/`, all `.png` files to `png/`, and the rest to `other/`. Show the result with `tree`. Why do `IMG_014.JPG` and `holiday.jpeg` not end up in `jpg/`?
10. **(6)** Use `echo` to show which arguments `ls docs/*` would receive. Why is `.settings` missing? Show it with `ls`.
11. **(6)** Back up the whole `docs/` directory to `backup/docs_copy`. Show that the hidden file was copied too, and compare the disk usage of both directories.
12. **(6)** Use `file` on `README.txt`, `src/hello.c`, `photos/jpg/img001.jpg` and `logs`, and `du -sh` on `data`, `logs`, `photos`, `src`. Why is `photos` now 16K although all photos are empty?
13. **(7)** Set the access and modification time of `docs/notes_01.txt` to 1 September 2026, 12:00. Show both times with `stat -c`, and explain why `ls -l` now shows a time of day and not a year.

Part C --- Challenge (28 points)

14. **(7)** Create a hard link `docs/syllabus_hard.txt` to `docs/syllabus.txt`. Show inode numbers and link counts. Delete `docs/syllabus.txt` and show that the data still exists. What is the link count now?

15. **(8)** Create a symbolic link `alpha_docs` in `~/lab` pointing to `projects/alpha/docs` and list its content through the link. Rename `projects/alpha` to `projects/alpha_old`: what happens to the link? Repair it.

16. **(6)** With *one* command create `course/week01` … `course/week14`, each containing `slides` and `notes`. How many directories were created in total (including `course`)? Show your calculation.

17. **(7)** Clean up: try `rmdir practice` and explain the result; then remove `practice` and `course` completely and list `~/lab`.

	Part A	Part B	Part C	Total
Points	33	39	28	100

15 Lab Sheet Solutions

Real output of `code/demo_lab_solutions.sh`, run in a fresh lab (inode numbers and times will differ on your machine).

Part A

1–2. `-p` creates the whole path; brace expansion gives three names to one `touch`:

```
student@okan-lab:~/lab$ mkdir -p practice/a/b/c
student@okan-lab:~/lab$ tree practice
practice
└── a
    └── b
        └── c

4 directories, 0 files
student@okan-lab:~/lab$ touch practice/{one,two,three}.txt
student@okan-lab:~/lab$ ls practice
a  one.txt  three.txt  two.txt
```

3–4. The inode number is the same before and after `mv`, because a rename only changes the directory entry; the inode and the data are not touched:

```
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/syllabus_backup.txt
student@okan-lab:~/lab$ ls -l backup
total 4
-rw-rw-r-- 1 student student 578 Oct 12 08:48 syllabus_backup.txt
student@okan-lab:~/lab$ ls -i backup/syllabus_backup.txt
804718 backup/syllabus_backup.txt
student@okan-lab:~/lab$ mv backup/syllabus_backup.txt backup/syllabus_v1.txt
student@okan-lab:~/lab$ ls -i backup/syllabus_v1.txt
804718 backup/syllabus_v1.txt
```

5–7. `students.csv` has 41 lines, one of them the header, so $41 - 1 = 40$ students:

```
student@okan-lab:~/lab$ head -n 4 docs/syllabus.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
Week 4  Threads and CPU scheduling basics
student@okan-lab:~/lab$ tail -n 2 docs/syllabus.txt
Week 14 Introduction to shell scripting
Week 15 Final exam
student@okan-lab:~/lab$ wc docs/linux_quotes.txt
  5  50 302 docs/linux_quotes.txt
student@okan-lab:~/lab$ wc -l data/students.csv
41 data/students.csv
student@okan-lab:~/lab$ cat "docs/final report.txt"
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ cat docs/final\ report.txt
Final report draft.
This file name contains a space.
```

Part B

8.

```
student@okan-lab:~/lab$ ls photos/img00?.jpg
photos/img001.jpg  photos/img004.jpg  photos/img007.jpg
photos/img002.jpg  photos/img005.jpg  photos/img008.jpg
photos/img003.jpg  photos/img006.jpg  photos/img009.jpg
student@okan-lab:~/lab$ ls photos/img01[0-2].jpg
photos/img010.jpg  photos/img011.jpg  photos/img012.jpg
student@okan-lab:~/lab$ echo photos/*.png
photos/img013.png
```

9. `IMG_014.JPG` is not matched by `*.jpg` because names are case-sensitive, and `holiday.jpeg` has a different extension; both are moved by the third `mv`. (A shorter alternative for the last step: `mv *.* other/`, since the three new directories contain no dot.)

```
student@okan-lab:~/lab$ cd photos
student@okan-lab:~/lab/photos$ mkdir jpg png other
student@okan-lab:~/lab/photos$ mv *.jpg jpg/
student@okan-lab:~/lab/photos$ mv *.png png/
student@okan-lab:~/lab/photos$ mv *.JPG *.jpeg other/
student@okan-lab:~/lab/photos$ tree
.
├── jpg
│   ├── img001.jpg
│   ├── ...
│   └── img012.jpg
├── other
│   ├── IMG_014.JPG
│   └── holiday.jpeg
└── png
    └── img013.png

4 directories, 15 files
student@okan-lab:~/lab/photos$ cd ..
```

10–11. `*` does not match names starting with a dot, so `.settings` is not among the arguments; `ls -a` shows it. `cp -r` copies everything, hidden files included (`ls -A` lists them without `.` and `..`); both trees use 32 K:

```
student@okan-lab:~/lab$ echo docs/*
docs/final report.txt docs/linux_quotes.txt docs/notes_01.txt docs/notes_02.txt docs/notes_03.txt docs/
```

```
     syllabus.txt
student@okan-lab:~/lab$ ls -a docs
 .    .settings           linux_quotes.txt   notes_02.txt   syllabus.txt
 ..  'final report.txt'   notes_01.txt       notes_03.txt
student@okan-lab:~/lab$ cp -r docs backup/docs_copy
student@okan-lab:~/lab$ ls -A backup/docs_copy
 .settings           linux_quotes.txt   notes_02.txt   syllabus.txt
'final report.txt'   notes_01.txt       notes_03.txt
student@okan-lab:~/lab$ du -sh docs backup/docs_copy
32K     docs
32K     backup/docs_copy
```

12. `photos/` now contains three subdirectories; each directory uses one 4 KiB block: $4 \times 4 = 16$ K, while the empty photo files still use 0 blocks:

```
student@okan-lab:~/lab$ file README.txt src/hello.c photos/jpg/img001.jpg logs
README.txt:            ASCII text
src/hello.c:           C source, ASCII text
photos/jpg/img001.jpg: empty
logs:                  directory
student@okan-lab:~/lab$ du -sh data logs photos src
16K     data
36K     logs
16K     photos
8.0K    src
```

13. `ls -l` prints the time of day because 1 September is less than six months before today:

```
student@okan-lab:~/lab$ touch -d "2026-09-01 12:00" docs/notes_01.txt
student@okan-lab:~/lab$ stat -c "%n | access %x | modify %y" docs/notes_01.txt
docs/notes_01.txt | access 2026-09-01 12:00:00.000000000 +0000 | modify 2026-09-01 12:00:00.000000000
    +0000
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student student 16 Sep  1 12:00 docs/notes_01.txt
```

Part C

14. Same inode, link count 2; after `rm` the count is 1 and the data is intact:

```
student@okan-lab:~/lab$ ln docs/syllabus.txt docs/syllabus_hard.txt
student@okan-lab:~/lab$ ls -li docs/syllabus.txt docs/syllabus_hard.txt
804121 -rw-rw-r-- 2 student student 578 Oct 12 08:48 docs/syllabus.txt
804121 -rw-rw-r-- 2 student student 578 Oct 12 08:48 docs/syllabus_hard.txt
student@okan-lab:~/lab$ rm docs/syllabus.txt
student@okan-lab:~/lab$ ls -li docs/syllabus_hard.txt
804121 -rw-rw-r-- 1 student student 578 Oct 12 08:48 docs/syllabus_hard.txt
student@okan-lab:~/lab$ head -n 2 docs/syllabus_hard.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
```

15. After the rename the link dangles (the stored path `projects/alpha/docs` no longer exists); remove it and create it again with the new path (or, in one step, `ln -sfn projects/alpha_old/docs alpha_docs`):

```
student@okan-lab:~/lab$ ln -s projects/alpha/docs alpha_docs
student@okan-lab:~/lab$ ls alpha_docs/
design.txt
student@okan-lab:~/lab$ mv projects/alpha projects/alpha_old
student@okan-lab:~/lab$ ls alpha_docs/
ls: cannot access 'alpha_docs/': No such file or directory
student@okan-lab:~/lab$ rm alpha_docs
student@okan-lab:~/lab$ ln -s projects/alpha_old/docs alpha_docs
```

```
student@okan-lab:~/lab$ ls -l alpha_docs
lrwxrwxrwx 1 student student 23 Oct 12 08:48 alpha_docs -> projects/alpha_old/docs
```

16. 1 (`course`) $+14$ (weeks) $+14 \times 2$ (`slides`, `notes`) $= 43$ directories:

```
student@okan-lab:~/lab$ mkdir -p course/week{01..14}/{slides,notes}
student@okan-lab:~/lab$ ls course
week01  week03  week05  week07  week09  week11  week13
week02  week04  week06  week08  week10  week12  week14
student@okan-lab:~/lab$ ls course/week10
notes  slides
```

17. `rmdir` refuses because `practice` is not empty; `rm -r` removes both trees (check the names with `ls` first):

```
student@okan-lab:~/lab$ rmdir practice
rmdir: failed to remove 'practice': Directory not empty
student@okan-lab:~/lab$ rm -r practice course
student@okan-lab:~/lab$ ls
README.txt  backup  docs  photos    scripts
alpha_docs  data    logs  projects  src
```

16 Looking Ahead: Week 11

Today we ignored the first column of `ls -l`, `-rw-rw-r--`, and the line `Access: (0664/-rw-rw-r--)` of `stat`. Next week, **Permissions, Ownership & Users**, explains them: users and groups (`id`, `/etc/passwd`), the read/write/execute bits for files and directories, `chmod` in symbolic and octal form (for example $754 = 7|5|4$ with $r = 4$, $w = 2$, $x = 1$), `umask`, `chown` and `sudo`. All of these are stored in the same inode we used today.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Chapter 13 (File-System Interface) and Chapter 14 (File-System Implementation).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Chapter 39, “Interlude: Files and Directories” (hard and symbolic links, `rename`, `unlink`, `stat`): `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
- W. Shotts, *The Linux Command Line*, Chapters 4 (Manipulating Files and Directories) and 7 (Seeing the World as the Shell Sees It), free at `https://linuxcommand.org/tlcl.php`.